

Logseq DB-Version Import

Implementation plan — stakeholder brief

Plan: docs/superpowers/plans/2026-08-20-logseq-db-import.md · Spec: docs/research-logseq-db-format.md · 2026-08-20

Decision: import Logseq 2.0 “DB version” graphs via the official EDN `:graph export` only — never raw `db.sqlite` — then transpile the parsed graph to synthetic Logseq-file markdown and feed it through the existing, tested file-import pipeline unchanged.

PHASES PLANNED 6 0 in progress	NEW RUST DEP 1 EDN parser crate	NEW DART DEPS 0 reuses FFI bridge	SCHEMA MAJOR GATE 65 ≠ 65 rejected
---	--	--	---

Executive summary

- **Decision:** consume the official EDN `:graph export` only. Transpile each page to synthetic Logseq-file markdown (`key:: value, - TODO [#A], [[Title]], journals/YYYY_MM_DD.md`) and feed the existing `VaultDialect::Logseq` pipeline — new code is the EDN reader and transpiler, nothing else.
- **Why:** `db.sqlite`’s `kvs` table stores transit-JSON-encoded index-tree nodes of a **forked** datascript — not queryable by SQL, and coupled to Logseq’s fork internals. The EDN export is schema-version-stamped and explicitly built as the machine-stable surface, against roughly monthly schema churn (65.7 → 65.33: 26 minor bumps in 13 months).
- **Scope:** exactly one new Rust dependency (the EDN parser), zero new Dart dependencies. Parsing, Typst assembly, dedupe, and the import UI are all reused as-is.
- **Status:** all 6 phases are planned; none started. Each phase carries one measurable exit metric — see the scorecard below.
- **Key risk:** the transpiler’s whole fixture rests on inferred EDN key names. Phase 0 requires a ground-truth diff against one real export before any later phase builds on it.

Why EDN, never SQLite

Storage layer	<code>kvs (addr, content, addresses)</code> — <code>content</code> is a transit-JSON-encoded datascript index-tree node, not a row-per-datom table; not usefully queryable with plain SQL.
Fork risk	Logseq pins a fork of datascript plus custom transit handlers (<code>Entity</code> , <code>ExceptionInfo</code> , <code>js/Error</code>). Upstream datascript’s storage is not guaranteed byte-compatible.
Schema churn	Schema 65.7 (Jul 2025) → 65.33 (Aug 2026): 26 minor migrations in about 13 months, roughly monthly. Major has held at 65 through the beta.
Machine surface	EDN <code>:graph export</code> : schema-version-stamped, documented as “designed to be simple, reliable and for machines,” with a supported round-trip (<code>graph import --type edn</code>).

Vendor’s own stability ranking

EDN export >> reading via `nbb-logseq/deps/db` (tracks the fork) >> raw SQLite parsing (last resort). The plan couples to the first tier only.

Phase plan

Phase	Exit metric	Status
0. Fixture & ground truth	Fixture parses; every construct — journal, task, typed property, class tag, uuid ref, asset — present exactly once and asserted.	Planned
1. EDN graph model	Fixture + one real export load with 0 errors; a wrong-major export is rejected with a named error.	Planned
2. Transpiler	100% of fixture blocks appear in output; flatten counters match a hand-count; snapshot test green.	Planned
3. Pipeline wiring	<code>convert_db_export()</code> runs end-to-end with uuid-based note IDs; all emitted notes <code>typst</code> compile clean.	Planned
4. App integration	UI import of the fixture graph succeeds on macOS; assets resolve.	Planned
5. Real-graph trial & release	0 compile failures; 0 unexplained duplicate ids; dropped-construct report written.	Planned

Key risks

EDN key-shape drift

The Phase-0 fixture is hand-written against key names **inferred** from source reading (`:pages-and-blocks`, `:build/children`, `:build/properties`, `:build/tags`, `:build/journal`, `status/priority` ids). Task 1 Step 2 requires diffing those against one real `export-edn` output before Task 2 starts — the fixture is the contract every later phase builds on, so this check is not optional.

Lossy flattening is counted, not silent

Typed property values, closed values, the class/property ontology, and DB-only views cannot round-trip to file-Logseq markdown. Global constraint: every lossy flattening increments a counter surfaced in the import report — never a silent drop.

Deliberate non-goals

- Reading `db.sqlite/kvs` directly — blocked by the global constraints; revisit only if Logseq drops the EDN export.
- Importing the class/property ontology as TyLog structure — counted, not modeled; add later if users ask.
- Live sync with a running Logseq 2.0 app over its HTTP API — out of scope; import is one-shot.
- `search-db.sqlite`, `client-ops-db.sqlite` — derived/sync data, ignored by design.